

MHIPEX — Full Research Project Blueprint

CLEF HIPE-2026 | Person–Place Relation Extraction

Researcher: Aman Jaiswal | Mentor: Dr. Sarika Jain, NIT Kurukshetra

Goal: Top-class research system + CLEF 2026 paper + open-source release

SECTION 1 — What You Are Building

You are building MHIPEX — a multilingual AI system that reads sentences from 19th–20th century historical newspapers and decides:

- `at` → Has this person *ever* been at this place historically?
- `isAt` → Is this person at this place *right now* (at publication time)?

Input: A sentence + person name + place name

Output: `at` = TRUE/FALSE and `isAt` = TRUE/FALSE

Languages: English · French · German · Luxembourgish

Tracks: Accuracy · Efficiency · Generalization

SECTION 2 — Datasets

2.1 Primary Dataset — HIPE-2026 (MANDATORY)

Item	Detail
Source	https://github.com/hipe-eval/HIPE-2026-data
Format	CoNLL-style TSV with entity annotations
Languages	EN, FR, DE, LB (Luxembourgish)
Split	Train / Dev / Test
Labels	<code>at</code> , <code>isAt</code> binary per entity pair
Domain	19th–20th century newspaper archives

How to access:

```
git clone https://github.com/hipe-eval/HIPE-2026-data.git
```

2.2 Supporting Datasets (for pretraining + transfer)

Dataset	Use	Link
HIPE-2022	Transfer learning + NER pretraining	github.com/hipe-eval/HIPE-2022-data
HIPE-2020	Historical multilingual NER baseline	github.com/hipe-eval/HIPE-2020-data
Impresso Corpus	Raw historical text (preprocessing experiments)	impresso-project.ch
WikiData KG	Entity linking context enrichment	huggingface.co/datasets/Babelscape/wikineural
mC4 (multilingual)	Language model warm-up if needed	huggingface.co/datasets/mc4

2.3 Dataset Statistics (Expected)

Language	Training Pairs	Dev Pairs	Test Pairs
English	~2,000	~400	~400
French	~2,000	~400	~400
German	~1,500	~300	~300
Luxembourgish	~500	~100	~100

Exact numbers confirmed after data access. Plan for class imbalance — isAt=TRUE will be rarer.

SECTION 3 — Tech Stack

Core Stack (Install First)

```
# Create virtual environment
python -m venv mhipex-env
source mhipex-env/bin/activate # Linux/Mac
# mhipex-env\Scripts\activate # Windows

# Core ML
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/wheels
pip install transformers datasets tokenizers accelerate
pip install scikit-learn numpy pandas tqdm

# NLP utilities
pip install spacy
python -m spacy download en_core_web_sm
python -m spacy download fr_core_news_sm
python -m spacy download de_core_news_sm

# RAG stack
pip install chromadb langchain sentence-transformers
pip install ollama # local LLM inference
```

```
# Experiment tracking
pip install wandb

# Distillation + Efficiency
pip install optimum onnxruntime

# Evaluation
pip install seqeval

# Development
pip install jupyter notebook black isort pytest
```

Model Checkpoints to Download

These are the exact Hugging Face model IDs you will use

```
MODELS = {
    "backbone": "xlm-roberta-base",          # Your main model
    "backbone_large": "xlm-roberta-large",    # Stronger comparison
    "mbert": "bert-base-multilingual-cased",  # Baseline
    "camembert": "camembert-base",           # French SOTA
    "hmbert": "dbmdz/bert-base-historic-multilingual-cased", # Historical SOTA
    "mdeberta": "microsoft/mdeberta-v3-base", # Modern SOTA comparison
    "embeddings": "sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
    "llm_fallback": "mistral:7b-instruct"     # Ollama local LLM
}
```

SECTION 4 — Project Folder Structure

```
mhipex/
├── data/
│   ├── raw/
│   │   ├── hipe2026/      ← git cloned HIPE-2026 data
│   │   ├── hipe2022/      ← HIPE-2022 data
│   │   └── hipe2020/      ← HIPE-2020 data
│   ├── processed/
│   │   ├── train.jsonl    ← preprocessed training pairs
│   │   ├── dev.jsonl
│   │   └── test.jsonl
│   └── chromadb/          ← vector store for RAG fallback
├── src/
│   ├── preprocessing/
│   │   ├── __init__.py
│   │   ├── ocr_fix.py     ← OCR noise cleaning
│   │   ├── spell_norm.py  ← archaic spelling normalization
│   │   ├── temporal_tag.py ← temporal cue extraction (spaCy)
│   │   └── build_dataset.py ← combine raw → processed JSONL
│   ├── models/
│   │   └── __init__.py
```

```

├── classifier.py      ← XLM-RoBERTa dual-head RE model
├── distillation.py   ← student model training
├── rag_fallback.py   ← ChromaDB + Ollama fallback
├── training/
│   ├── train.py      ← main training script
│   ├── evaluate.py   ← evaluation + metrics
│   └── predict.py    ← inference on test set
├── utils/
│   ├── metrics.py    ← macro-recall, F1 calculator
│   ├── logger.py     ←
│   └── config.py     ← all hyperparameters in one place
├── configs/
│   ├── base.yaml     ← default config
│   ├── accuracy_track.yaml ← Accuracy Profile config
│   ├── efficiency_track.yaml ← Efficiency Profile config (distilled)
│   └── generalization.yaml ← Generalization Profile config
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_baseline_experiments.ipynb
│   ├── 03_xlmr_finetuning.ipynb
│   ├── 04_ablation_studies.ipynb
│   └── 05_error_analysis.ipynb
├── experiments/
│   └── runs/          ← W&B / local experiment logs
├── results/
│   ├── predictions/  ← official HIPE format predictions
│   └── tables/        ← CSV result tables for paper
├── paper/
│   ├── main.tex      ← CLEF 2026 working notes paper
│   └── figures/       ← pipeline diagram, result charts
├── tests/
│   ├── test_preprocessing.py
│   ├── test_classifier.py
│   └── test_evaluation.py
├── requirements.txt
├── README.md
└── .gitignore

```

SECTION 5 — Core Model Code

5.1 Dual-Head Classifier (src/models/classifier.py)

```
import torch
import torch.nn as nn
from transformers import AutoModel, AutoTokenizer

class MHIPEXClassifier(nn.Module):
    """
    XLM-RoBERTa with two independent binary heads:
    - Head 1: at      (has person ever been at place?)
    - Head 2: isAt    (is person at place at publication time?)
    """
    def __init__(self, model_name="xlm-roberta-base", dropout=0.1):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(model_name)
        hidden = self.encoder.config.hidden_size # 768 for base, 1024 for large

        self.dropout = nn.Dropout(dropout)
        self.head_at = nn.Linear(hidden, 2) # binary: at TRUE/FALSE
        self.head_isat = nn.Linear(hidden, 2) # binary: isAt TRUE/FALSE

    def forward(self, input_ids, attention_mask, token_type_ids=None):
        outputs = self.encoder(
            input_ids=input_ids,
            attention_mask=attention_mask,
            token_type_ids=token_type_ids
        )
        cls = self.dropout(outputs.last_hidden_state[:, 0, :])
        return {
            "at_logits": self.head_at(cls),
            "isat_logits": self.head_isat(cls)
        }
```

5.2 Input Format

```
def build_input(sentence, person, place, tokenizer, max_length=256):
    """
    Format: [CLS] sentence [SEP] [ENT1: person] [SEP] [ENT2: place] [SEP]
    """
    text = f"[CLS] {sentence} [SEP] [ENT1: {person}] [SEP] [ENT2: {place}] [SEP]"
    return tokenizer(
        text,
        max_length=max_length,
        truncation=True,
        padding="max_length",
        return_tensors="pt"
    )
```

5.3 Training Loop

```
def train_epoch(model, loader, optimizer, scheduler, device):
    model.train()
    criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
    total_loss = 0

    for batch in loader:
        optimizer.zero_grad()
        outputs = model(
            batch["input_ids"].to(device),
            batch["attention_mask"].to(device)
        )
        loss_at = criterion(outputs["at_logits"], batch["at_labels"].to(device))
        loss_isat = criterion(outputs["isat_logits"], batch["isat_labels"].to(device))
        loss = loss_at + loss_isat  # equal weight on both heads

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()

    return total_loss / len(loader)
```

5.4 Evaluation Metric (Macro-Recall)

```
from sklearn.metrics import recall_score

def macro_recall(at_true, at_pred, isat_true, isat_pred):
    """
    HIPE-2026 primary metric: Macro-averaged Recall across both relations.
    """
    recall_at = recall_score(at_true, at_pred, average="macro")
    recall_isat = recall_score(isat_true, isat_pred, average="macro")
    return (recall_at + recall_isat) / 2
```

SECTION 6 — SOTA Comparison Plan

Run ALL of these on the same dev set before your final system:

Step	Model	Command
1	mBERT zero-shot	<code>--model bert-base-multilingual-cased --mode zero_shot</code>
2	mBERT fine-tuned	<code>--model bert-base-multilingual-cased --mode finetune</code>
3	XLM-R base fine-tuned	<code>--model xlm-roberta-base --mode finetune</code>
4	XLM-R large fine-tuned	<code>--model xlm-roberta-large --mode finetune</code>
5	hmBERT fine-tuned	<code>--model dbmdz/bert-base-historic-multilingual-cased</code>

Step	Model	Command
6	CamemBERT (FR only)	<code>--model camembert-base --lang fr</code>
7	mDeBERTa fine-tuned	<code>--model microsoft/mdeberta-v3-base --mode finetune</code>
8	GPT-4 zero-shot	exploratory only, NOT main benchmark
9	HIPE-2022 top systems	results from published papers
10	MHIPEX (yours)	full pipeline with RAG fallback

This table becomes Table 2 in your paper — the most important table.

SECTION 7 — Implementation Roadmap

Week 1 (May 20–26): Setup + Data

- ☐ Clone HIPE-2026 data repo
- ☐ Set up Python environment (requirements.txt)
- ☐ Run `01_data_exploration.ipynb` — understand label distribution, language splits
- ☐ Build preprocessing pipeline (ocr_fix, spell_norm, temporal_tag)
- ☐ Convert raw data to training JSONL format
- ☐ Log to W&B — set up project dashboard

Week 2 (May 27 – Jun 2): Baselines

- ☐ mBERT zero-shot run → record Macro-Recall
- ☐ mBERT fine-tuned run → record
- ☐ XLM-R base fine-tuned → record
- ☐ Save all baseline results to `results/tables/baselines.csv`
- ☐ Run `02_baseline_experiments.ipynb`

Week 3 (Jun 3–9): Main Model

- ☐ XLM-R large fine-tuned
- ☐ hmBERT fine-tuned → this is your strongest baseline to beat
- ☐ mDeBERTa fine-tuned
- ☐ MHIPEX full pipeline (Stage 1 + Stage 2)
- ☐ Run ablation: remove preprocessing → see drop in Macro-Recall
- ☐ Run `03_xlmr_finetuning.ipynb`

Week 4 (Jun 10–16): Efficiency + Generalization Track

- ☐ Knowledge distillation (6-layer student from XLM-R large teacher)
- ☐ INT8 quantization → measure latency speedup
- ☐ ChromaDB vector store setup from training data
- ☐ Ollama Mistral-7B setup for RAG fallback
- ☐ Test RAG fallback on dev set literary examples
- ☐ Submit to all 3 HIPE-2026 evaluation profiles

Week 5–6 (Jun 17–30): Paper

- ☐ Error analysis — where does the model fail? (language? entity type? sentence length?)
- ☐ Complete result tables (Table 1, 2, 3)
- ☐ Write CLEF 2026 working notes paper (8–10 pages, LNCS format)
- ☐ Submit paper + system output to HIPE-2026 by deadline

SECTION 8 — Hyperparameters to Tune

```
# configs/accuracy_track.yaml
model_name: "xlm-roberta-large"
max_length: 256
batch_size: 32
learning_rate: 2.0e-5
warmup_ratio: 0.1
num_epochs: 10
early_stopping_patience: 3
early_stopping_metric: "macro_recall"
dropout: 0.1
weight_decay: 0.01
optimizer: "adamw"
scheduler: "cosine_with_warmup"
class_weights_at: [1.0, 2.5] # upweight TRUE class if imbalanced
class_weights_isat: [1.0, 3.0] # isAt=TRUE will be rarer
seed: 42
```

SECTION 9 — Ablation Studies

Run these experiments to show in your paper which component matters most:

Ablation	What You Remove	Expected Effect
No preprocessing	Raw OCR text, no cleaning	↓ ~5–10% Macro-Recall
No temporal tagging	Remove time cue features	↓ ~3–6% (mostly on isAt)
Single head (at only)	Remove isAt head	N/A by design, show separately

Ablation	What You Remove	Expected Effect
No class weighting	Equal weights	↓ on minority class
Base vs Large	XLM-R base vs large	Δ Macro-Recall
No RAG fallback	Direct prediction only	↓ on Generalization track

SECTION 10 — Expected Results Table (Paper Table 2)

Model	EN Recall	FR Recall	DE Recall	LB Recall	Macro-Recall
mBERT zero-shot	~0.52	~0.48	~0.50	~0.41	~0.48
mBERT fine-tuned	~0.61	~0.57	~0.59	~0.49	~0.57
XLM-R base	~0.67	~0.64	~0.63	~0.54	~0.62
XLM-R large	~0.72	~0.69	~0.68	~0.58	~0.67
hmBERT	~0.71	~0.70	~0.72	~0.61	~0.69
mDeBERTa	~0.73	~0.70	~0.69	~0.59	~0.68
MHIPEX (yours)	~0.76	~0.73	~0.74	~0.64	~0.72

These are targets. Real numbers depend on your training. If hmBERT beats yours → analyze why and improve.

SECTION 11 — Paper Structure (CLEF 2026 Working Notes)

Title: MHIPEX: Multilingual Person-Place Relation Extraction
from Historical Texts with Knowledge-Grounded Transformers

1. Introduction (1 page)
2. Task Description – HIPE-2026 overview (0.5 page)
3. System Description (2 pages)
 - 3.1 Preprocessing pipeline
 - 3.2 XLM-RoBERTa dual-head classifier
 - 3.3 Efficiency track (distillation + quantization)
 - 3.4 Generalization track (RAG fallback)
4. Experimental Setup (0.5 page)
5. Results (1.5 pages)
 - Table 1: Ablation study
 - Table 2: SOTA comparison
 - Table 3: Per-language breakdown
6. Error Analysis (0.5 page)
7. Conclusion and Future Work (0.5 page)
- References

SECTION 12 — Key Commands (Quick Reference)

```
# Setup
git clone https://github.com/hipe-eval/HIPE-2026-data.git data/raw/hipe2026
pip install -r requirements.txt

# Preprocess data
python src/preprocessing/build_dataset.py \
    --input data/raw/hipe2026 \
    --output data/processed

# Train baseline
python src/training/train.py \
    --config configs/accuracy_track.yaml \
    --model bert-base-multilingual-cased

# Train MHIPEX
python src/training/train.py \
    --config configs/accuracy_track.yaml \
    --model xlm-roberta-large

# Evaluate
python src/training/evaluate.py \
    --model_path experiments/runs/best_model \
    --split dev

# Run RAG fallback
python src/models/rag_fallback.py \
    --build_index data/processed/train.jsonl \
    --test_file data/processed/test.jsonl

# Generate predictions for HIPE submission
python src/training/predict.py \
    --model_path experiments/runs/best_model \
    --output results/predictions/accuracy_track.tsv
```

SECTION 13 — How to Impress Dr. Jain

1. Send a weekly update email every Sunday with: what you ran, what the numbers were, what you are doing next. Short, clear, one table.
2. Keep all experiments in W&B — share the dashboard link with her once you have results.
3. Commit code to GitHub daily — she can see your progress in real time.
4. Beat hmBERT — this is the domain-specific SOTA. If your system beats it even slightly, that is a publishable result.
5. Write the error analysis section well — professors love when students explain *why* the model fails, not just the numbers.
6. Label every result clearly — never show a table without model name, dataset, metric, and date.

Blueprint version: May 2026 / MHIPEX / Built for CLEF HIPE-2026 Accuracy + Efficiency + Generalization Tracks